What Do NFMs See? Analyzing Attack Classification in Encrypted Traffic*

Yeasul Kim and Hwankuk Kim
Kookmin University, Seoul, Republic of Korea
{gimsul80, rinyfeel}@kookmin.ac.kr

Abstract

Conventional AI-based intrusion detection systems (AI-IDS) have been criticized for their vulnerability to shortcut learning due to reliance on human-defined features. Network Foundation Models (NFMs) offer the potential to overcome this limitation by directly processing packet information within flows in a structured manner. However, high performance alone cannot reveal what information within flows a model actually relies on for classification. In this paper, we classify 6 attack types in encrypted traffic using two NFMs that differ in how they represent traffic—netFound and NetMamba—and analyze what information each model relies on through packet/burst order shuffle experiments. netFound processes up to 72 packets organized in burst units, using tokenized headers along with burst-level metadata (bytes, IAT, flow duration, etc.) as input, whereas NetMamba uses only the header (80 bytes) and payload (240 bytes) of the first 5 packets. Both models achieved over 99% accuracy; however, the shuffle experiments revealed entirely different dependency structures. netFound showed no performance change even when all orders were shuffled, indicating reliance on flow-level metadata such as bytes per burst, inter-burst IAT, and flow duration. In contrast, NetMamba's accuracy dropped sharply to 52.79 ± 1.20% when packet order was shuffled, demonstrating strong dependence on packet sequence order. Notably, NetMamba exhibited the largest performance degradation for full-flow attacks (HTTP_CC, SQLi, Slowloris)—attacks distinguished by characteristics such as transmission intervals and data sizes that appear after the first 5 packets—yet NetMamba cannot directly observe these characteristics and was instead relying on the order information among the first 5 packets. This study highlights the importance of verifying not only the performance of NFMs in attack classification, but also what information the models rely on for classification.

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 18, July 8–10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

1 Introduction

Conventional AI-based intrusion detection systems (AI-IDS) rely on a supervised learning structure using human-defined features and labels. There is no consensus on feature engineering, performance is inconsistent across datasets, and it remains uncertain whether high performance reflects genuine learning of intrinsic attack characteristics. Jacobs et al. [1] experimentally demonstrated that the high performance of existing AI-IDS was in fact dependent on incidental data characteristics (byte alignment differences, IP TTL values, TCP flag patterns, etc.) rather than intrinsic attack characteristics. This reveals that existing AI-IDS are vulnerable to shortcut learning, where models become underspecified and rely on incidental data characteristics rather than the intrinsic characteristics of attacks.

In an environment where encrypted traffic continues to increase, AI-based attack detection must assume encryption and be able to verify what traces attack-type-specific characteristics leave in packets and flows. NFMs offer potential in this direction. They can pre-train general-purpose traffic representations through self-supervised learning on large volumes of unlabeled traffic, then be applied to attack detection through fine-tuning with small amounts of labeled data, preserving flow structure as much as possible by processing packet headers and payloads within flows in a structured manner. However, NFMs also cannot reveal what information within flows the model relies on for detection based on high performance alone, and the insight that "models cannot be trusted based on performance alone" applies equally to NFMs.

In this paper, we classify 6 attack types in encrypted traffic using two NFMs with different traffic representation methods—netFound [2] and NetMamba [3]—and analyze what information each model relies on through packet/burst order shuffle experiments. The contributions of this study are as follows:

- First, we performed encrypted traffic attack type classification using two NFMs with different traffic representation methods, selecting and analyzing 6 attack types based on where attack characteristics are revealed within flows (early-flow vs. full-flow).
- Second, through packet/burst order shuffle experiments, we confirmed that behind the identical 99% classification performance, the two models rely on entirely different information. netFound maintained 99% performance even when all orders were shuffled, relying on flow-level metadata (bytes per burst, IAT, etc.), whereas NetMamba's accuracy dropped sharply to 52.79 ± 1.20% when packet order was shuffled, demonstrating strong dependence on packet order.
- Third, for NetMamba, the intrinsic characteristics of full-flow attacks (HTTP_CC, SQLi, Slowloris)—such as transmission intervals and data sizes—appear after the first 5 packets, meaning NetMamba cannot learn these characteristics and was instead relying on packet order information. This highlights the importance of verifying not only performance but also what information models rely on for classification in NFM-based attack detection.

2 Background: Network Foundation Model (NFM)

2.1 Beyond AI-IDS: The Move Toward NFM

Conventional AI-based intrusion detection systems (AI-IDS) rely on a supervised learning pipeline in which human-defined features and labels are used to classify network traffic. This approach is inherently limited by the scope of features that humans can design, and the process of compressing flows into statistical features causes the loss of sequential structure within flows.

Model Property	NetMamba (2024)	netFound (2023)
Architecture	Mamba	Transformer
Complexity	$O(n)$	$O(n^2)$
Max Packets	5	72
Packet header	80 bytes	24 bytes
Packet payload	240 bytes	12 bytes
Metadata	-	Flow-level: flow_duration, protocol; Burst-level: directions, bytes, counts, iats

Table 1: Comparison of Input Processing Between netFound and NetMamba

Consequently, these models suffer from poor generalization when applied to different datasets, which is directly linked to the shortcut learning problem discussed in Section 1 [1].

NFMs represent a paradigm shift from this approach. By leveraging self-supervised learning on large volumes of unlabeled traffic data, NFMs learn general-purpose representations of network traffic, which can then be fine-tuned with small amounts of labeled data for various downstream tasks such as application identification, attack detection, and traffic classification. The key difference from conventional AI-IDS is that NFMs preserve packet-level information and the sequential structure of flows by processing the header and payload data of packets within a flow in a structured manner. However, the way each model represents traffic differs, and this is the core subject of analysis in this study.

We first define key terms commonly used across NFM research. A flow is a set of bidirectional packets sharing the same 5-tuple (source/destination IP, source/destination port, and protocol), representing a single communication session. A burst is a group of consecutive packets within a flow that share the same direction. For example, consecutive packets from client to server form a single burst. The specific criteria for burst segmentation may vary across models (e.g., netFound groups consecutive packets within 10ms into a single burst). These two concepts serve as the fundamental units by which NFMs structure and process traffic, and this paper adopts the same definitions.

The learning scope of NFMs has progressively expanded. PERT [4] learned byte-level patterns from a fixed range of encrypted payload within individual packets. ET-BERT [5] also used a fixed range of encrypted payload but introduced the burst concept to capture bidirectional data exchange patterns within flows. However, both models rely solely on encrypted payload and do not consider packet header information—such as TCP flags, ports, and window size—where traces of attacks may be present. The subsequently proposed netFound [2] and NetMamba [3] explicitly incorporate packet headers, adopting traffic representation methods suitable for attack detection as a downstream task. In this study, we select these two models as our analysis targets, and their specific input processing methods are detailed in Section 2.2.

Attempts to diagnose whether a model genuinely learns the intrinsic characteristics of attacks have also been made in prior AI-IDS research. TRUSTEE [1] diagnosed shortcut learning, in which a model relies on incidental data characteristics rather than the intrinsic nature of attacks. In the NFM domain, a recent study [6] analyzed the latent representations of pre-trained NFMs through Embedding Geometry Analysis to assess representation space utilization, Metric Alignment Assessment to measure correspondence with domain-expert features, and Causal Sensitivity Testing to evaluate robustness to protocol perturbations. In contrast, this study focuses on the attack detection task at the fine-tuning stage. That is, unlike prior AI-IDS diagnosis (TRUSTEE) or the analysis of pre-trained NFM representations, we directly verify whether NFMs actually learn and utilize per-attack-type traces for classification during fine-tuning. To this end, we employ a simple diagnostic based on packet/burst order shuffling.

Algorithm 1: netFound Preprocessing (pcap → Arrow record)

Input: pcap file of a single flow (packets sharing the same 5-tuple)
Output: flow record R

```
group packets into bursts                          # consecutive same-direction
packets = 1 burst
for each burst b_i:
    for each packet in b_i:
      h ← 24 header bytes used in tokenization
          # IPv4: {HeaderLen, ToS, TotalLen, Flags, TTL}
          # TCP: {Flags, WinSize, SeqNum, AckNum, UrgentPtr}
          # UDP: {Length} / ICMP: {Type, Code}
        p ← first 12 bytes of payload
        packet_tokens ← group concat(h, p) into 2-byte units   # 36 bytes → 18 tokens
    burst_tokens[i] ← list of packet_tokens in b_i             # all packets of burst i
    directions[i], bytes[i], counts[i], iats[i] ← burst-level metadata
flow_duration ← last_packet_timestamp – first_packet_timestamp
protocol ← L4 protocol
R ← {burst_tokens, directions, bytes, counts, iats, flow_duration, protocol, label}
return R
```

2.2 netFound and NetMamba

In this study, we use two NFMs that differ fundamentally in their architecture and input processing methods.

netFound [2] is a Transformer-based NFM that processes flow as bidirectional sessions. It accommodates up to 72 packets per flow, and for each packet, 24 bytes of header fields (the IPv4, TCP, UDP, and ICMP header fields defined in [2]; see Algorithm 1) and 12 bytes of payload are grouped into 2-byte units, yielding 18 tokens per packet. A distinctive feature of netFound is that it explicitly includes rich flow-level metadata as input, beyond packet-level information. It provides the model with each burst's direction (directions), total bytes per burst (bytes), total packet count per burst (counts), inter-burst arrival time intervals (iats), as well as flow-level total duration (flow_duration) and protocol (protocol). This enables the model to directly utilize the metadata of the flow. The preprocessing procedure for converting the raw pcap into the input format (an Arrow record) is summarized in Algorithm 1.

NetMamba [3] is an NFM based on the Mamba architecture, a State Space Model, with O(n) computational complexity that can efficiently process long sequences. Using only the first 5 packets of a flow, and unlike netFound which selectively extracts protocol header fields, NetMamba tokenizes the raw bytes of a fixed length from a fixed offset without considering header fields. Each packet's 320 bytes (80 header + 240 payload) are arranged into an 8×40 row block, and the first 5 packets are stacked vertically to represent a single flow as a 40×40 matrix (a grayscale image). Unlike netFound, it does not provide burst-level metadata or flow-level information separately. The only information available to the model is the header and payload of the first 5 packets of the flow. The preprocessing procedure for converting the raw pcap into the input format (a 40×40 matrix, i.e., a grayscale image) is summarized in Algorithm 2.

The input processing methods and differences between the two models are summarized in Table 1. Even when receiving the same flow as input, netFound utilizes packet, burst, and flow-level

Algorithm 2: NetMamba Preprocessing (pcap → PNG image)

Input: pcap file of a single flow
Output: flow image I ∈ ℤ^(40 × 40)

```
P ← first 5 packets of the flow        # zero-pad if fewer than 5
for j = 0 to 4:
    pkt ← P[j]                         # j-th packet
    strip frame (Ethernet) header from pkt
     h ← 80 raw bytes from start of IP header      # cut if longer, zero-pad if shorter
     p ← 240 raw bytes from start of payload       # cut if longer, zero-pad if shorter
     I[8j:8j+8] ← reshape(concat(h, p), (8, 40))   # 320 bytes → 8 rows × 40 cols
return I
```

information collectively, whereas NetMamba uses only the first 5 packets of the flow. This difference in traffic representation affects which information each model relies on when classifying attacks, and this is concretely revealed through the shuffle experiments in Section 4.

3 Dataset & Attack Characteristics

3.1 Selection of Six Attack Types

In this study, we used a non-public dataset constructed by KISTI and KAIST. This dataset comprises 13 attack types consisting of encrypted traffic, where all attacks were conducted between the same attacker IP and target IP across various ports, allowing the characteristics of each attack type to be clearly distinguished on a per-flow basis under controlled conditions. Raw pcap files are provided, enabling bidirectional packets sharing the same 5-tuple to be grouped into individual flows, which makes flow-level model training suitable for NFMs. Datasets that simultaneously provide encrypted traffic, raw pcap files, and diverse attack types are scarce, and this dataset is well-suited for the purpose of this study in meeting these conditions.

From the 13 attack types, we selected 6. The selection criterion was the position within the flow where attack indicators are revealed.

- **Three attack types with indicators present in early flow packets (within the first 5 packets):**
  - ✓ **SYN Flood**: A DoS attack that exhausts server connection resources by sending a large volume of packets with the SYN flag set. TCP flags such as SYN and RST themselves serve as direct attack indicators.
  - ✓ **PortScan**: A reconnaissance attack that probes for open ports on the target. Flows consist of 2 packets (FIN → RST+ACK), with the abnormal TCP FIN flag serving as an indicator.
  - ✓ **BruteSSH**: A brute-force password attack against SSH accounts. Immediately after the TCP 3-way handshake, the SSH protocol banner (SSH-2.0) is exchanged in plaintext, making it distinguishable from other attacks within the first 5 packets.

Attack Type	Protocol	Flow Packet Exchange Pattern (first 5 packets)	Remaining (median-5)
SYNFlood	TCP	(A→T) SYN, (T →A) SYN+ACK, (A→T) FIN	-
Port Scan	TCP	(A→T) FIN, (T →A) RST+ACK	-
BruteSSH	TCP +SSH	(A→T) SYN, (T →A) SYN+ACK, (A→T) ACK, (A→T) Client Protocol [SSH-2.0], (T →A) ACK	~1pkts
Slowloris	TCP +TLS1.3	(A→T) SYN, (T →A) SYN+ACK, (A→T) ACK, (A→T) Client Hello [TLS1.3], (T →A) ACK	~13pkts
SQLi	TCP +TLS1.3	(A→T) SYN, (T →A) SYN+ACK, (A→T) ACK, (A→T) Client Hello [TLS1.3], (T →A) ACK	~7pkts
HTTP CC	TCP +TLS1.3	(A→T) SYN, (T →A) SYN+ACK, (A→T) ACK, (A→T) Client Hello [TLS1.3], (T →A) ACK	~11pkts

Table 2: Flow Scenario per Attack Type (A: Attacker, T: Target)

- **Three attack types with indicators revealed across the entire flow:**

  The following three attacks share nearly identical first 5 packets (SYN → SYN+ACK → ACK → Client Hello → ACK), and their differences are revealed through flow-level metadata.

  - ✓ **Slowloris:** A DoS attack that occupies server connections for extended periods by intentionally sending incomplete HTTP request headers at slow intervals. Characterized by long IATs and small bytes per burst.
  - ✓ **SQLi**: An attack that injects malicious SQL statements to exfiltrate database information. Characterized by exceptionally long inter-burst IATs and large bytes per burst.
  - ✓ **HTTP_CC**: A DDoS attack that exhausts server resources by rapidly sending large volumes of HTTP requests that resemble legitimate traffic. Characterized by very short inter-burst IATs.

For this experiment, 12,000 flows per attack type were used, totaling 72,000 flows. Table 2 summarizes the first 5 packet exchange patterns for each attack, showing that the three early-indicator attacks reveal attack evidence within the packets themselves, while the three full-flow attacks have nearly identical initial packets. Furthermore, packet sizes and TCP flags are also similar, making it difficult to distinguish the three full-flow attacks from their initial packets alone.

Table 3 presents the flow metadata statistics per attack type. Although the three full-flow attacks share identical initial packets, they exhibit clear differences in flow-level metadata. SQLi has the largest inter-burst IAT (mean 23.20s) and the largest bytes per burst (mean 1,402). Slowloris has the longest flow duration (mean 9.40s) and the smallest median bytes per burst (median 104). HTTP_CC has the smallest median inter-burst IAT (median 0.02s), reflecting its characteristic of rapidly sending repeated requests. These metadata differences are directly related to the information that netFound uses as input (bytes, iats, flow_duration, etc.), and serve as key evidence for interpreting the shuffle experiment results that follow.

Stat	SYN Flood	Port Scan	Brute SSH	Slowloris	SQLi	HTTP CC
Packets per flow						
mean	3	2	7.4	18.3	12.4	15.9
median	3	2	6	18	12	16
max	5	3	23	19	18	16
Flow duration						
mean	0.001	0.001	0.65	**9.40**	2.27	1.50
median	0.001	0.001	0.002	5.00	0.01	0.03
max	7.80	0.001	58.78	54.84	58.10	55.19
Inter-burst IAT						
mean	-	-	1.65	3.30	**23.20**	1.42
median	-	-	1.87	0.70	20.37	**0.02**
max	-	-	16.97	54.84	58.09	55.16
Bytes per burst						
mean	319	40	384	487	**1402**	968
median	44	40	152	**104**	1283	841
max	1104	80	1776	1906	2161	2161
Total bytes per flow						
mean	638	80	1007	3713	3077	3980
median	640	80	340	3714	2914	4005
max	1148	124	4141	3819	4409	4112

Table 3: Flow Metadata Statistics per Attack Type - mean / median / max

3.2 Experimental Setup

Classification Experiment. Fine-tuning experiments were conducted to evaluate the attack type classification performance of the two NFMs on encrypted traffic. The experiments were performed as multi-class classification across 6 attack types. Normal traffic was not included, as the purpose of this study is not to determine whether traffic is an attack or not (binary classification of normal/attack), but to analyze what information within flows NFMs rely on when classifying attack types. By focusing on attack type classification, we can more clearly observe what characteristics of each attack the models use to distinguish between them.

A total of 72,000 flows were used, with 12,000 flows per attack type, split into 70% for training and 30% for testing. Both models were used with the publicly available GitHub code [7, 8] and pre-trained models published on Hugging Face [9, 10]. During fine-tuning, the hyperparameters recommended in each paper (e.g., learning rate) were applied, and the number of epochs was set to 4 for both models. To verify the consistency of results, experiments were repeated 3 times by changing the random seed (0, 123, 7) for the train-test split. The experiments were conducted on a Linux 18.04 system with an AMD Ryzen 9 9950X 16-Core Processor (CPU), 64GB RAM, and an NVIDIA RTX A6000 (48GB VRAM, GPU).

Shuffle Experiment. Both models achieved over 99% accuracy in the classification experiment; however, this performance alone cannot reveal what information within flows the models actually rely on for classification. To investigate this, we designed shuffle experiments in which the packet/burst order of the test data was shuffled and inference was performed using the already-trained models. If performance is maintained after shuffling, the model does not rely on sequence order information; if performance degrades, the model depends on sequence order information. The shuffle experiments were also repeated 3 times with different random seeds to verify consistency.

- **NetMamba shuffle experiment:** The order of the first 5 packets in each flow was randomly shuffled. Since NetMamba processes the information of 5 packets sequentially, this experiment verifies whether the model relies on packet sequence order. The detailed shuffle procedure is presented in Algorithm 3.

Algorithm 3: NetMamba Packet-order Shuffle

Input: flow image I (40×40), where packet j occupies rows [8j:8j+8]
Output: shuffled flow image I'

```
for j = 0 to 4:
    P[j] ← I[8j:8j+8]                    # extract the j-th packet block (8 rows)
order ← random_permutation(5)            # random ordering of the 5 packets, per flow
for j=0 to 4:
    I'[8j:8j+8] ← P[order[j]]            # place packets in the shuffled order
return I'
```

- **netFound shuffle experiment:** Since netFound structures flows in burst units, shuffle experiments were conducted under the following 3 conditions:
  - ✓ **Inter-burst shuffle**: Burst order is shuffled while packet order within bursts is preserved.
  - ✓ **Intra-burst shuffle:** Burst order is preserved while packet order within bursts is shuffled.
  - ✓ **Both shuffled:** Both burst order and packet order within bursts are shuffled.

These 3 conditions allow us to separately verify whether netFound relies on inter-burst order, intra-burst packet order, or both. The detailed shuffle procedure is presented in Algorithm 4.

Algorithm 4: netFound Burst-based Shuffle

Input: flow record R with bursts B = $[b_1, \ldots, b_k]$ # k = number of bursts
condition ∈ {INTER, INTRA, BOTH}
Output: shuffled record R'
packet tokens and burst metadata are preserved; only order changes

```
if condition ∈ {INTRA, BOTH}:                    # shuffle packets within each burst
    for each burst b_i:
        n_i ← number of packets in b_i
        if |n_i| > 1:
            pkt_order ← random_permutation(|n_i|)    # new ordering of packets in b_i
            reorder packets of b_i by pkt_order
if condition ∈ {INTER, BOTH}:                    # shuffle burst order
    if k > 1:
        burst_order ← random_permutation(k)      # new ordering of the k bursts
        reorder (burst_tokens, directions, bytes, counts, iats) jointly by burst_order
return R'
```

4 Results & Analysis

4.1 Classification Results

Both models achieved over 99% accuracy in classifying the 6 attack types, with consistent performance across 3 repeated experiments. Table 4 summarizes the classification accuracy of both models across 3 runs, showing stable performance in the range of 99.88–99.93% regardless of seed. While both models demonstrated near-perfect classification performance, a small number of misclassifications were observed. Table 5 presents the major misclassification patterns observed across the 3 runs, showing the common misclassification directions and their frequencies in both models.

Model	Seed 0	Seed 7	Seed 123	Mean ± Std
netFound	0.9992	0.9992	0.9991	0.9992 ± 0.0001
NetMamba	0.9992	0.9988	0.9993	0.9991 ± 0.0003

Table 4: Classification Accuracy (3 runs)

A notable finding is the common misclassification direction observed in both models. The tendency for full-flow attacks Slowloris and HTTP_CC to be misclassified toward BruteSSH appeared consistently across all 3 runs, regardless of the model. All three attacks share a similar structure with initial packets beginning with a TCP 3-way handshake, but BruteSSH differs in that the SSH protocol banner exists in plaintext. Additionally, sporadic misclassifications were observed, including SQLi→Slowloris(Seed 123, 1 case) in netFound, and BruteSSH→PortScan (Seed 0, 2 cases), SQLi→BruteSSH(Seed 0, 1 case), and PortScan→SYNFlood(Seed 123, 1 case) in NetMamba; however, all were limited to 1–2 cases and did not form a consistent pattern.

The fact that both models achieved identical performance of over 99% alone cannot reveal what information within flows each model actually relies on for classification. To investigate this, Section 4.2 analyzes the results of the shuffle experiments.

Model	Misclassification Direction	Seed 0	Seed 7	Seed 123
netFound	Slowloris → BruteSSH	4	5	6
	HTTP_CC → BruteSSH	13	12	12
NetMamba	Slowloris → BruteSSH	4	2	4
	HTTP_CC → BruteSSH	11	23	10

Table 5: Misclassification Patterns (3 runs)

4.2 Shuffle Experiment Results

netFound Shuffle Results. Table 6 summarizes the accuracy of netFound under 4 shuffle conditions. Inter-burst shuffle showed no performance change across all 3 runs, and the both shuffled results were identical to the intra-burst shuffle, confirming that netFound does not utilize inter-burst order for classification at all. Intra-burst shuffle also showed no change for seed 0 and seed 7; however, for seed 123 only, SQLi classification dropped from 3,588 to 2,518 correct classifications, resulting in an accuracy of 0.9487. This suggests that netFound may be partially sensitive to intra-burst packet order under certain data splits.

In summary, netFound does not rely on inter-burst order and is largely independent of intra-burst packet order. The information netFound primarily relies on is burst-level metadata (bytes, counts, iats,

directions, flow_duration, protocol). As confirmed in Table 3, this metadata exhibits clear differences across attack types, enabling netFound to distinguish the 6 attack types without sequence information.

This insensitivity of netFound to burst/packet ordering is an expected consequence of its design philosophy regarding traffic input representation. netFound is designed with an emphasis on flow-level metadata and on extracting protocol header field-based information from packets. The shuffle in our experiments only changes the order of bursts and packets, while the values of this metadata and the protocol header fields themselves remain unchanged. Therefore, netFound's insensitivity to ordering can be partly attributed to this input representation.

Shuffle Condition	Seed 0	Seed 7	Seed 123	Mean ± Std
Original *	0.9992	0.9992	0.9991	0.9992 ± 0.0001
Inter-burst shuffle	0.9992	0.9991	0.9992	0.9992 ± 0.0001
Intra-burst shuffle	0.9992	0.9989	0.9487	0.9823 ± 0.0291
Both shuffled	0.9992	0.9989	0.9487	0.9823 ± 0.0291

Table 6: netFound Shuffle Experiment Results (Accuracy, **same as Table 4*)

NetMamba Shuffle Results. Table 7 summarizes the accuracy before and after packet order shuffling for NetMamba, and Figure 1 presents the per-attack correct classification counts after shuffling.

Shuffle Condition	Seed 0	Seed 7	Seed 123	Mean ± Std
Original *	0.9992	0.9988	0.9993	0.9991 ± 0.0003
Shuffled	0.5153	0.5393	0.5290	0.5279 ± 0.0120

Table 7: NetMamba Shuffle Experiment Results (Accuracy, **same as Table 4*)

After shuffling the order of the 5 packets, accuracy dropped sharply from 99.91 ± 0.03% to 52.79 ± 1.20%, and this pattern was consistent across all 3 runs. The impact of shuffling varied markedly across attack types. BruteSSH was perfectly maintained across all 3 runs (3,600/3,600), and SYN Flood was also maintained above 90% (3,273–3,535). This is because distinctive indicators exist within the packets themselves, such as the SSH banner (SSH-2.0) and TCP flags (SYN, RST), enabling classification without sequence order. In contrast, HTTP_CC (584–613) and SQLi (409–469) collapsed most severely, and Slowloris (1,455–1,742) also declined substantially. These three attacks are TLS-based with nearly identical headers and payloads in their first 5 packets, and misclassifications after shuffling were concentrated toward BruteSSH.

The key finding is as follows. The full-flow attacks (HTTP_CC, SQLi, Slowloris) are attacks whose distinguishing characteristics appear after the first 5 packets. As confirmed in Table 3, HTTP_CC sends rapidly repeated requests with short inter-burst IAT (median 0.02s), SQLi sends large requests at long intervals with long inter-burst IAT (mean 23.20s) and large bytes per burst (mean 1,402), and Slowloris sends small data slowly with long flow duration (mean 9.40s) and small bytes per burst (median 104). However, since NetMamba only receives the first 5 packets as input, it cannot directly observe these attack characteristics. The 99% classification performance before shuffling was not the result of learning the intrinsic characteristics of these attacks, but rather heavily relied on the order information among the first 5 packets, as revealed through the shuffle experiment.

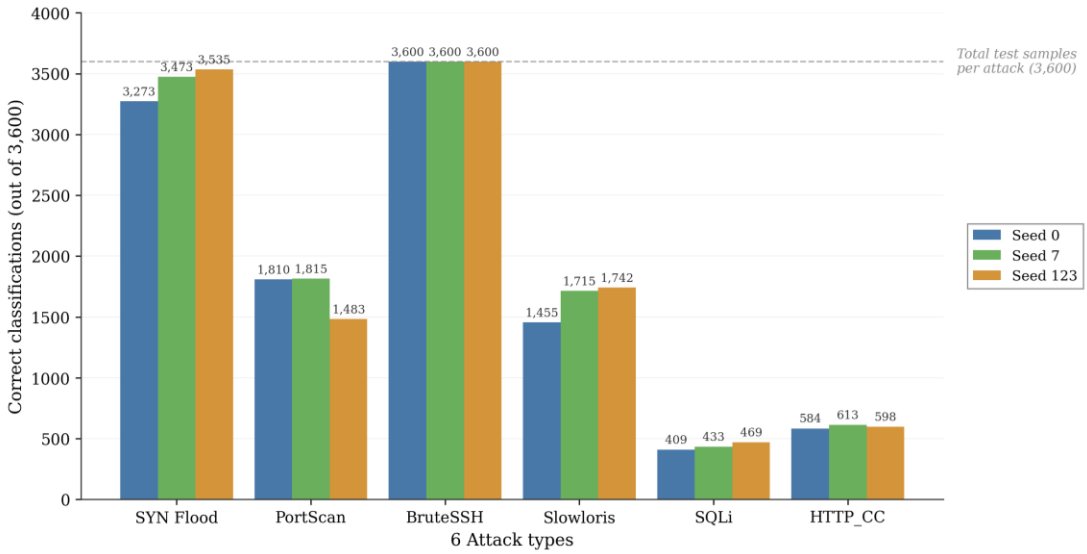


Figure 1: NetMamba per-attack correct classifications after packet order shuffle (3 seeds)

5 Discussion & Conclusion

In this study, we classified 6 attack types in encrypted traffic using two NFMs with different traffic representation methods—netFound and NetMamba—and analyzed what information each model relies on within flows through shuffle experiments.

Both models achieved over 99% classification accuracy; however, the shuffle experiments revealed that the two models rely on entirely different information. netFound maintained its performance even when all orders were shuffled, relying on flow-level metadata, whereas NetMamba's accuracy dropped sharply to 52.79 ± 1.20% when packet order was shuffled. In particular, NetMamba had not learned the intrinsic characteristics of full-flow attacks (HTTP_CC, SQLi, Slowloris) but was instead relying on the order information of the first 5 packets. These results demonstrate that models cannot be trusted based on performance alone in NFM-based attack classification, and that verifying what information models rely on for classification is essential.

This study confirmed that NFMs hold the potential to overcome the shortcut learning limitations of conventional AI-IDS by preserving and processing flow structure, while also revealing through shuffle experiments that performance alone cannot explain the basis of a model's classification. Future research plans to address current limitations and expand the experiments.

First, the shuffle experiments in this study only verified dependence on sequence order information. More detailed ablation studies—such as masking or removing specific header fields or metadata—are needed to more precisely isolate the information models depend on. Second, since this study used 6 of the 13 attack types in the dataset, we plan to extend the analysis to all attack types for further validation. Third, this study analyzed only two NFMs. However, various NFMs have recently emerged, including NetMamba+ [11], which supplements NetMamba's limitations by adding meta-information such as IAT and packet length, as well as TrafficFormer [12] and YaTC [13], which processes traffic by visualizing it as images. Future work plans to expand experiments with these models to more comprehensively analyze attack classification dependency structures according to NFM traffic representation methods.

Finally, since the results of this study were obtained on simulated attack traffic from a single controlled environment, validation in other network environments is needed. In addition, this study focused on classification among attack types and did not include normal traffic, so its conclusions should be understood as limited to classification among attack types. Nevertheless, the difference in the dependency structures of the two models arises from the difference in input representation discussed in Section 2.2, and is a characteristic of NFMs stemming from their traffic input representation design, independent of the dataset. Verifying whether the findings of this study are reproduced on public datasets or traffic from diverse environments, and analyzing how the models' dependency structures change in settings that include normal traffic, are directions that can be addressed in future work.

Acknowledgement

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.RS-2023-00235509, Development of security monitoring technology-based network behavior against encrypted cyber threats in ICT convergence environment).

References

[1] Jacobs, A. S., Beltiukov, R., Willinger, W., Ferreira, R. A., Gupta, A., & Granville, L. Z. (2022, November). AI/ML for network security: The emperor has no clothes. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (pp. 1537–1551). https://doi.org/10.1145/3548606.3560609

[2] Guthula, S., Beltiukov, R., Battula, N., Guo, W., & Gupta, A. (2023). netFound: Foundation model for network security (Version 4) [Preprint]. arXiv. https://arxiv.org/abs/2310.17025v4

[3] Wang, T., Xie, X., Wang, W., Wang, C., Zhao, Y., & Cui, Y. (2024, October). NetMamba: Efficient network traffic classification via pre-training unidirectional Mamba. In 2024 IEEE 32nd International Conference on Network Protocols (ICNP) (pp. 1–11). IEEE. https://doi.org/10.1109/ICNP61940.2024.10858569

[4] He, H. Y., Yang, Z. G., & Chen, X. N. (2020, December). PERT: Payload encoding representation from transformer for encrypted traffic classification. In 2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K) (pp. 1–8). IEEE. https://doi.org/10.23919/ITUK50268.2020.9303204

[5] Lin, X., Xiong, G., Gou, G., Li, Z., Shi, J., & Yu, J. (2022, April). ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In Proceedings of the ACM Web Conference 2022 (pp. 633–642). https://doi.org/10.1145/3485447.3512217

[6] Beltiukov, R., Guthula, S., Guo, W., Willinger, W., & Gupta, A. (2025). Demystifying network foundation models. Advances in Neural Information Processing Systems, 38.

[7] SNL-UCSB. (2023). netFound [Source code]. GitHub. https://github.com/SNL-UCSB/netFound

[8] Wang, T. (2024). NetMamba [Source code]. GitHub. https://github.com/wangtz19/NetMamba

[9]SNL-UCSB. (2023). netFound-640M-base [Pretrained model]. Hugging Face. https://huggingface.co/snlucsb/netFound-640M-base

[10] Wang, T. (2024). *NetMamba* [Pretrained model]. Hugging Face. https://huggingface.co/wangtz/NetMamba

[11] Wang, T., Xie, X., Wang, W., Wang, C., Liu, J., Huang, B., Hu, Y., Zhao, Y., & Cui, Y. (2026). NetMamba+: A framework of pre-trained models for efficient and accurate network traffic classification [Preprint]. arXiv. https://arxiv.org/abs/2601.21792

[12] Zhou, G., Guo, X., Liu, Z., Li, T., Li, Q., & Xu, K. (2025, May). TrafficFormer: An efficient pre-trained model for traffic data. In 2025 IEEE Symposium on Security and Privacy (SP) (pp. 1844–1860). IEEE. https://doi.org/10.1109/SP61157.2025.00102

[13] Zhao, R., Zhan, M., Deng, X., Wang, Y., Wang, Y., Gui, G., & Xue, Z. (2023, June). Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation. Proceedings of the AAAI Conference on Artificial Intelligence, 37(4), 5420–5427. https://doi.org/10.1609/aaai.v37i4.25674